

Airbnb Open Dataset – Data Cleaning, EDA & Insights Report

1. Project Overview

This project focuses on **cleaning, preprocessing, and analyzing the Airbnb Open Dataset** to extract meaningful insights related to pricing, reviews, neighbourhoods, and property characteristics. The workflow follows a **real-world data analytics pipeline**, starting from raw data ingestion to feature engineering and exploratory data analysis (EDA).

The objective of this analysis is to:

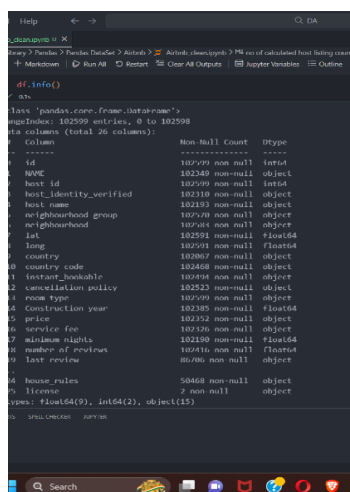
- Clean raw Airbnb listing data
- Handle missing values using statistical and logical methods
- Remove duplicates and irrelevant features
- Perform exploratory data analysis (EDA)
- Generate pricing, review, and location-based insights
- Answer real business questions using data

2. Dataset Overview

- **Source:** Airbnb Open Dataset
- **Initial Rows:** 102,599
- **Data Types:** Numerical, Categorical, Date-Time, Text

Key columns include:

- Listing details (price, room type, minimum nights)
- Host information (host name, host verification)
- Location details (neighbourhood, neighbourhood group)
- Review metrics (reviews per month, review rate number)



```
df.info()
Out[1]:
<class 'pandas.core.frame.DataFrame'>
Int64Index: 102599 entries, 0 to 102598
Data columns (total 26 columns):
 #   Column                Non-Null Count  Dtype  ---
 0   id                    102599 non-null  int64  ---
 1   name                 102549 non-null  object  ---
 2   host_id              102599 non-null  int64  ---
 3   host_identity_verified 102110 non-null  object  ---
 4   host_name            102493 non-null  object  ---
 5   neighbourhood_group   102478 non-null  object  ---
 6   neighbourhood         102581 non-null  object  ---
 7   lat                  102591 non-null  float64 ---
 8   long                 102591 non-null  float64 ---
 9   country              102867 non-null  object  ---
10  country_code         102468 non-null  object  ---
11  instant_bookable     102494 non-null  object  ---
12  cancellation_policy   102523 non-null  object  ---
13  room_type            102509 non-null  object  ---
14  construction_year     102385 non-null  float64 ---
15  price                102352 non-null  object  ---
16  service_fee          102126 non-null  object  ---
17  minimum_nights       102190 non-null  float64 ---
18  number_of_reviews     102416 non-null  float64 ---
19  last_review          102486 non-null  object  ---
20  house_rules          58468 non-null  object  ---
21  license              7 non-null      object  ---
dtypes: float64(9), int64(2), object(15)
```



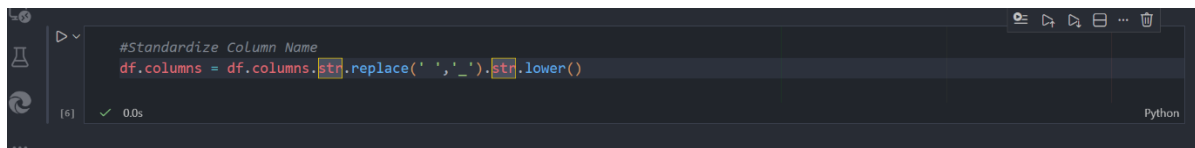
```
df.isnull().sum()
Out[1]:
id                    0
name                 250
host_id              0
host_identity_verified 289
host_name            406
neighbourhood_group   20
neighbourhood         16
lat                   8
long                  8
country              42
country_code         131
instant_bookable     105
cancellation_policy   76
room_type            10
construction_year     214
price                247
service_fee          273
minimum_nights       409
number_of_reviews     183
last_review          19293
reviews_per_month    15879
review_rate_number     426
calculated_host_listings_count 318
availability_365      448
house_rules          52131
license              102597
dtype: int64
```

3. Data Cleaning & Preprocessing

3.1 Column Standardization

- Converted column names to lowercase
- Replaced spaces with underscores

Purpose: Ensure consistency and easier column access during analysis.

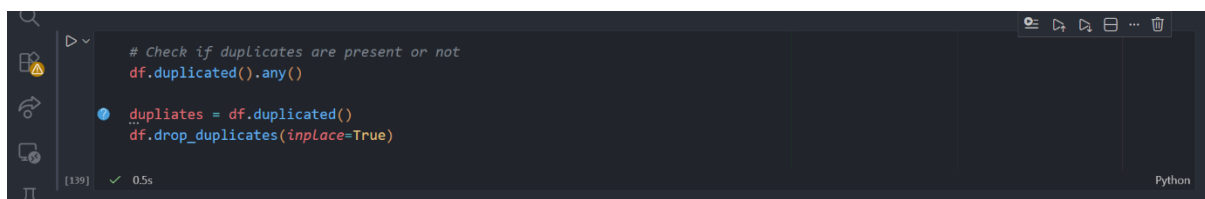


```
#Standardize Column Name
df.columns = df.columns.str.replace(' ', '_').str.lower()
```

[6] ✓ 0.0s Python

3.2 Duplicate Handling

- Identified duplicate rows using `df.duplicated()`
- Removed duplicate records to maintain data integrity



```
# Check if duplicates are present or not
df.duplicated().any()

duplicates = df.duplicated()
df.drop_duplicates(inplace=True)
```

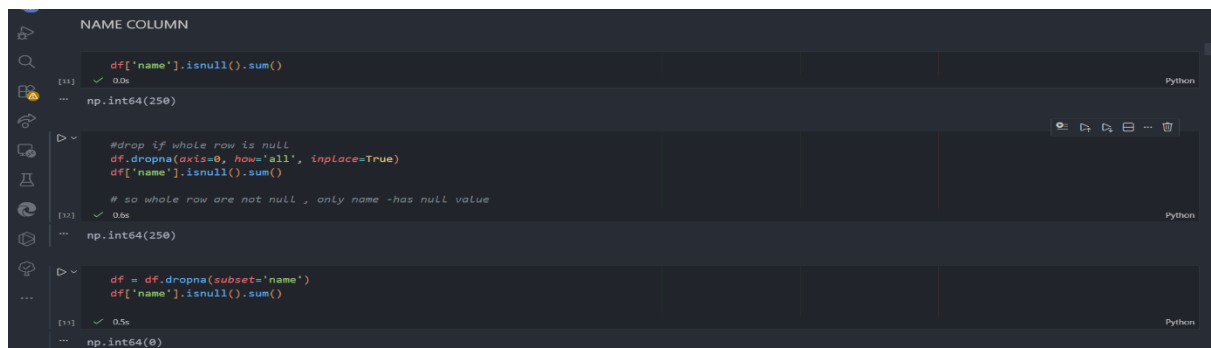
[139] ✓ 0.5s Python

3.3 Missing Value Treatment

Different strategies were applied depending on column importance:

a) Critical Identifiers

- Rows with missing **listing names** were removed



```
NAME COLUMN

df['name'].isnull().sum()
np.int64(250)

#drop if whole row is null
df.dropna(axis=0, how='all', inplace=True)
df['name'].isnull().sum()

# so whole row are not null , only name -has null value
np.int64(250)

df = df.dropna(subset='name')
df['name'].isnull().sum()
np.int64(0)
```

[11] ✓ 0.0s Python

[12] ✓ 0.0s Python

[13] ✓ 0.5s Python

b) Host Information

- `host_identity_verified`, `host_name` → filled with 'Not Given'

```
df['host_identity_verified'] = df['host_identity_verified'].fillna('Not Given')
```

```
df['host_name'] = df['host_name'].fillna('Not Given')
```

c) Location Columns

- Fixed spelling inconsistencies in `neighbourhood_group`
- Removed rows with missing neighbourhood and neighbourhood group values

```
df['neighbourhood_group'] = df['neighbourhood_group'].str.replace('brookln', 'Brooklyn')
df['neighbourhood_group'] = df['neighbourhood_group'].str.replace('manhatan', 'Manhattan')

# remove rows where there is null value in neighbourhood group
df = df.dropna(subset='neighbourhood_group')
```

d) Geographic Coordinates

- Removed rows with missing latitude values

e) Country Information

- Filled missing country with 'United States'
- Filled missing country code with 'US'

```
df['country'] = df['country'].fillna('United States')
df['country'].isna().sum()

np.int64(0)
```

f) Booking Attributes

- Removed rows with missing `instant_bookable`
- Filled `minimum_nights` using **mode** grouped by **neighbourhood group** and **room type**

```
df = df.dropna(subset='instant_bookable')
df['instant_bookable'].isna().sum()

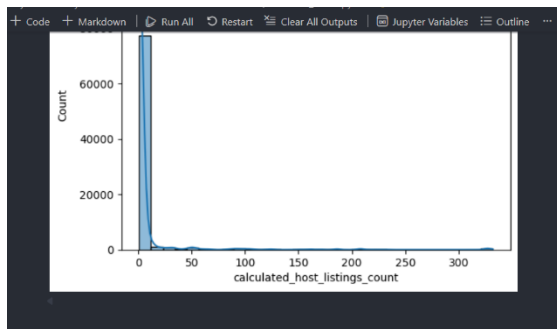
np.int64(0)
```

```
df['minimum_nights'] = df.groupby(['neighbourhood_group', 'room_type'])['minimum_nights'].transform(lambda x: x.fillna(x.mode()[0]))
```

g) Calculated host listings

```
df['calculated_host_listings_count'].value_counts()
df['calculated_host_listings_count'].skew()
```

np.float64(7.971732695881438)



Skewness is 7.97 which tells data is heavily right skewed.

So null values are handled with median.

```
#skew is 7.97 which mean data is heavily right skewed , so we are filling null values with median
df['calculated_host_listings_count'] = df['calculated_host_listings_count'].fillna(df['calculated_host_listings_count'].median())
df['calculated_host_listings_count'].skew()
```

np.float64(7.984139640207758)

h) Review Data

- reviews_per_month filled with 0
- Missing review_rate_number filled with **mean value**

```
df['reviews_per_month'].isnull().sum()
df['reviews_per_month'] = df['reviews_per_month'].fillna(0)
```

```
df['review_rate_number'].skew()
...
Interpretation:
Skew = 0 -> symmetric
Skew > 1 -> highly right-skewed
Skew < -1 -> highly left-skewed
...
np.float64(-0.1366382572359661)

# skewness is -0.13 which is close to 0, we can assume that data is almost normally distributed
r_mean = df['review_rate_number'].mean()
df['review_rate_number'] = df['review_rate_number'].fillna(r_mean)
```

3.4 Data Type Conversion

- Remove null values in construction year
- Converted construction_year to DateTime
- Cleaned and converted price to numeric
- Removed currency symbols and commas from numeric columns

```
df = df.dropna(subset='construction_year')

df['construction_year'] = pd.to_datetime(df['construction_year'], errors='coerce', format='%Y')
```

```
df['price'] = df['price'].str.replace(',', '')
df['price'] = df['price'].str.replace('$', '')

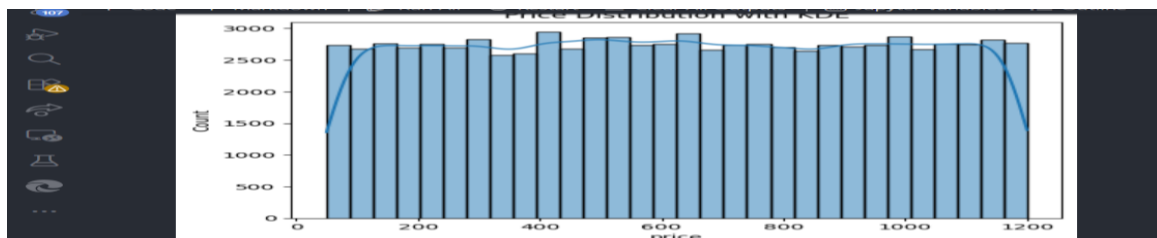
df = df.dropna(subset='price')
# df['price'] = pd.to_numeric(df['price'], errors='coerce')
df['price'] = df['price'].astype(int)
df['price'].isna().sum()
```

np.int64(0)

```
df['price'].skew()
```

np.float64(0.00024053853122929582)

Skewness is 0.00024 , which is almost price is normally distributed we can assume this.



3.5 Column Removal

Dropped irrelevant or redundant columns to simplify analysis:

- house_rules
- availability_365
- license
- latitude & longitude
- country_code
- host_id and listing id

3.6 Text & Numeric Standardization

- Trimmed whitespaces from string columns
- Removed special characters
- Cleaned numeric columns using regex-based filtering

```
Airbnb_clean.ipynb - X
Python Library > Pandas > Pandas DataSet > Airbnb > Airbnb_clean.ipynb > str_cols = df.select_dtypes(include='object').columns
+ Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ...

num_cols = df.select_dtypes(include=['number']).columns
import pandas as pd
import re

Tabnine | Edit | Test | Explain | Document
def clean_numeric(x):
    # Skip NaN
    if pd.isna(x):
        return x
    # Convert to string to remove unwanted characters
    x_str = str(x).strip()
    # Keep only digits, decimal point, minus sign
    x_str = re.sub(r'[^d\.-]', '', x_str)

    # Convert back to float
    try:
        return float(x_str)
    except:
        return pd.NA # or np.nan

# Apply only to numeric-like columns
df[num_cols] = df[num_cols].applymap(clean_numeric)
```

```
>
str_cols = df.select_dtypes(include='object').columns
df[str_cols] = df[str_cols].astype(str)
df[str_cols] = df[str_cols].apply(lambda x: (
    x.str.strip()
    .str.replace(r'\s+', ' ', regex=True)
    .str.replace(r'^a-zA-Z0-9 ', '', regex=True)
))
```

4. Exploratory Data Analysis (EDA)

4.1 Summary Statistics

- Used `df.describe()` to understand data distribution
- Removed invalid minimum nights (≤ 0)
- Capped minimum nights at 365 to reduce extreme outliers

EDA

```
import matplotlib.pyplot as plt
#a) Summary statistics
df.describe()
```

	construction_year	price	minimum_nights	number_of_reviews	reviews_per_month	review_rate_number	calculated_host_listings_count
count	85244	85244.000000	85244.000000	85244.000000	85244.000000	85244.000000	85244.000000
mean	2012-06-27 12:50:01.154333440	625.999953	7.437192	32.374736	1.378419	3.278972	7.015391
min	2003-01-01 00:00:00	50.000000	-365.000000	1.000000	0.000000	1.000000	1.000000
25%	2007-01-01 00:00:00	340.000000	2.000000	3.000000	0.220000	2.000000	1.000000
50%	2012-01-01 00:00:00	625.000000	3.000000	11.000000	0.745000	3.000000	1.000000
75%	2017-01-01 00:00:00	913.000000	5.000000	38.000000	2.010000	4.000000	2.000000
max	2022-01-01 00:00:00	1200.000000	5645.000000	1024.000000	90.000000	5.000000	332.000000
std	NaN	331.637193	28.001187	52.172479	1.752576	1.281350	29.421433

```
df = df[df['minimum_nights'] > 0 ]

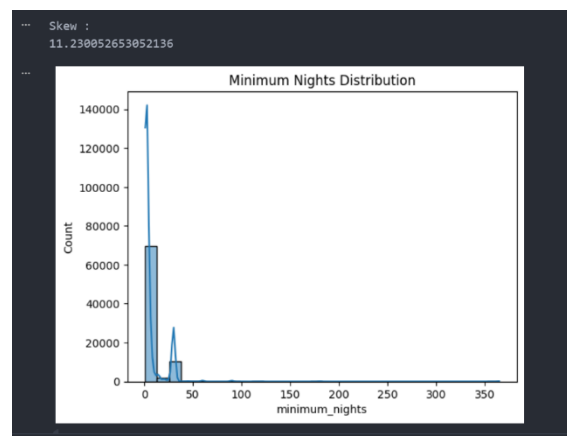
# Optional: cap very high minimum nights to 365 (1 year)
df.loc[df['minimum_nights'] > 365, 'minimum_nights'] = 365

# b) Distribution checks
print('Skew : ')
print(df['minimum_nights'].skew())

import seaborn as sns

sns.histplot(df['minimum_nights'], bins=30, kde=True)
plt.title('Minimum Nights Distribution')
plt.show()
```

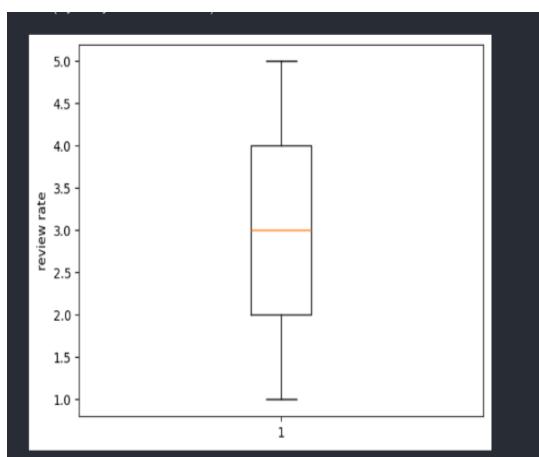
Skew :
11.230052653052136



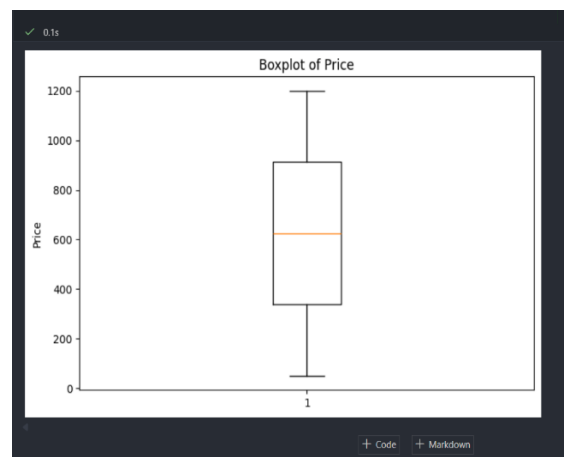
Insight:

Most listings prefer **short-term stays**, long stays are rare.

Review rate outlier detection using box plot :



Price outlier detection using box plot :



5. Feature Engineering

A. Extracted Year from Construction year

```
df['year'] = df['construction_year'].dt.year  
df['year']
```

✓ 0.0s

Business Insight Enabled:

- Identification of trends in property construction over time
- Understanding market growth and urban development patterns

B. Created Property Age

```
df['property_age'] = df['year'].max() - df['year']  
price_year = df.groupby('property_age')['price'].mean().astype(int)
```

Purpose:

- ➔ Measure how old each property is relative to the newest property
- ➔ Helps understand depreciation or premium effects in Airbnb pricing

C. Price Categorization

```
# Define labels for the bins  
labels = ['Low', 'Medium', 'High', 'Very High']  
  
# Map the bins to labels  
df['price_label'] = pd.qcut(df['price'], 4, labels=labels)  
price_review = df.groupby('price_label')['review_rate_number'].mean()  
price_review
```

Business Insight Enabled:

- Higher price categories did **not** result in higher review ratings
- Customer satisfaction is independent of price tier

6. Analysis

1. Deeper Price Analysis

A. Number of Host by room type

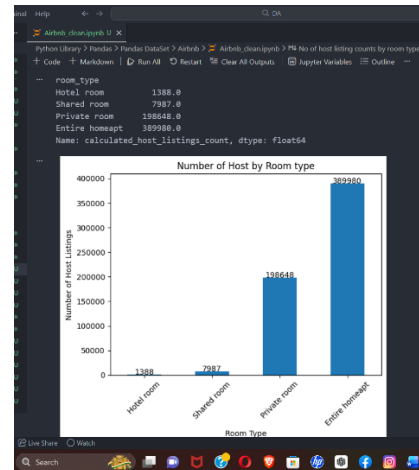
```
No of host listing counts by room type

count (variable) count: Series[Any] ulated_host_listings_count'].sum().sort_values()
print(count)

count.plot(kind='bar')
plt.title("Number of Host by Room type")
plt.xlabel('Room Type')
plt.ylabel('Number of Host Listings')
plt.xticks(rotation=45)

# Add values on top of bars
for i, v in enumerate(count):
    plt.text(i, v + 5, f'{v:.0f}', ha='center') # v+5 puts value slightly above bar
plt.show()

room_type
Hotel room      1388.0
Shared room     7987.0
Private room   198648.0
Entire homeapt 389980.0
Name: calculated_host_listings_count, dtype: float64
```



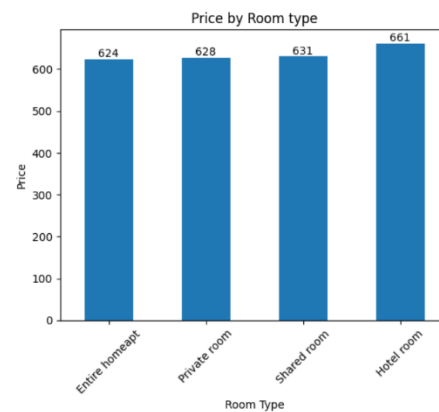
Insight : Most people prefer entire home apartment.

B. Price by Room type

```
Price by Room type

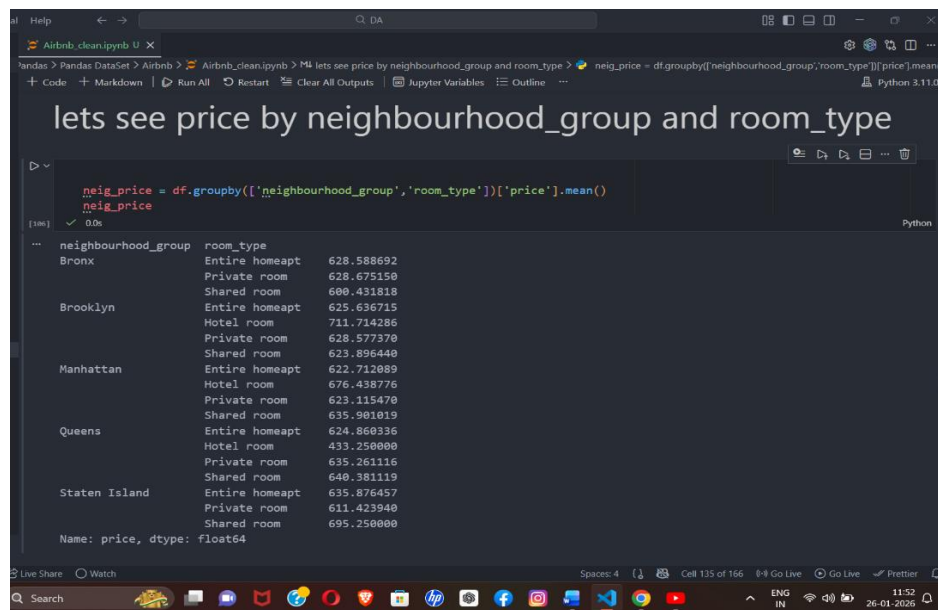
price_room_type = df.groupby('room_type')['price'].mean().sort_values()
print(price_room_type)

room_type
Entire homeapt    624.344885
Private room     627.620917
Shared room      630.694132
Hotel room       661.407080
Name: price, dtype: float64
```



Insight : Entire homes often cheaper compared to other available options

C. Price by Room type in each Neighbourhood Group

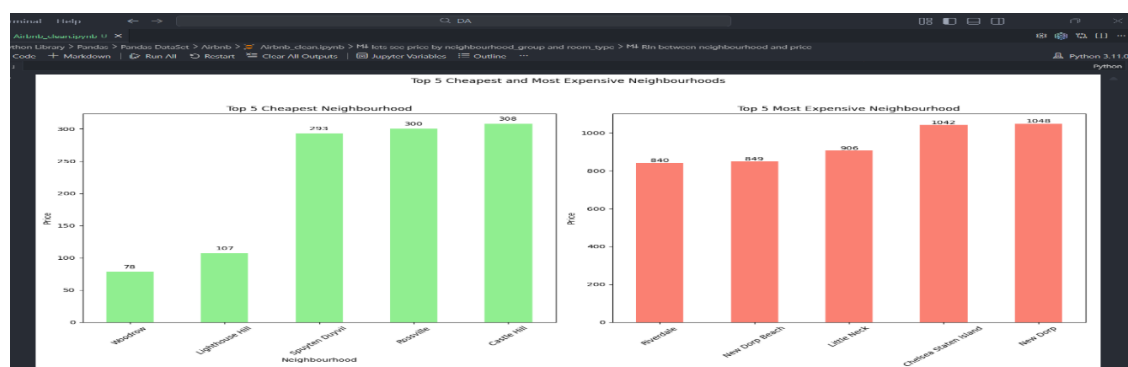


D. Price by Neighbourhood



Woodrow -> \$78 (Cheapest Neighbourhood)
New Dorp -> \$1048 (Expensive Neighbourhood)

Top 5 cheapest and expensive neighbourhood



Cheapest neighbourhood

Woodrow	\$78
Lighthouse Hill	\$107
Spuyten Duyvil	\$293
Rossville	\$300
Castle Hill	\$308

Expensive neighbourhood

Riverdale	\$840
New Dorp Beach	\$849
Little Neck	\$906
Chelsea Staten Island	\$1042
New Dorp	\$1048

E. Price by Neighbourhood group

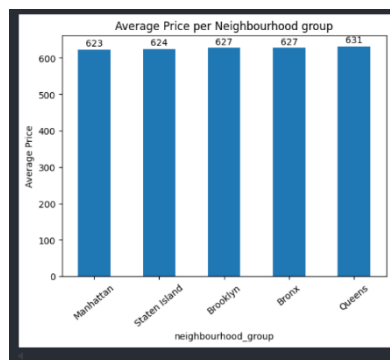
```

RIn between neighbourhood_group and price
>
avg_neighbourhood_group = df.groupby('neighbourhood_group')['price'].mean()
avg_neighbourhood_group.sort_values().astype(int)
print(avg_neighbourhood_group)

avg_neighbourhood_group.plot(kind='bar')
plt.ylabel('Average Price')
plt.title('Average Price per Neighbourhood group')
plt.xticks(rotation=40)

for i,v in enumerate(avg_neighbourhood_group):
    plt.text(i,v+10, f'{v}', ha='center')
plt.show()

```



Neighbourhood_group

Manhattan	\$623
Staten Island	\$624
Brooklyn	\$627
Bronx	\$627
Queens	\$631

Lets see In :

- Cheapest (**Manhattan**) neighbourhood group , top 5 cheapest and expensive neighbour.
- Expensive (**Queens**) neighbourhood group , top 5 cheapest and expensive neighbour.

```

#Cheapest group and most expensive neighbourhood group
cheap_ng = avg_neighbourhood_group.idxmin()
expensive_ng = avg_neighbourhood_group.idxmax()

print("Cheapest neighbourhood group:", cheap_ng)
print("Most expensive neighbourhood group:", expensive_ng)
print("-----")

#compute avg price per neighbourhood within neighbourhood group
## Cheapest and expensive 5 neighbourhoods in the cheapest group
cheap_group_df = df[df['neighbourhood_group'] == 'Brooklyn']
avg_price_brooklyn = cheap_group_df.groupby('neighbourhood')['price'].mean().sort_values()
cheap_top5_cheap = avg_price_brooklyn.head(5).astype(int)
cheap_top5_exp = avg_price_brooklyn.tail(5).astype(int)

## Cheapest and expensive 5 neighbourhoods in the expensive group
exp_group_df = df[df['neighbourhood_group'] == 'Queens']
avg_price_queens = exp_group_df.groupby('neighbourhood')['price'].mean().sort_values()
exp_top5_cheap = avg_price_queens.head(5).astype(int)
exp_top5_exp = avg_price_queens.tail(5).astype(int)

print("Top 5 cheapest neighbourhoods in (cheap_ng):")
print(cheap_top5_cheap)

print("Top 5 most expensive neighbourhoods in (cheap_ng):")
print(cheap_top5_exp)

print("Top 5 cheapest neighbourhoods in (expensive_ng):")
print(exp_top5_cheap)

```

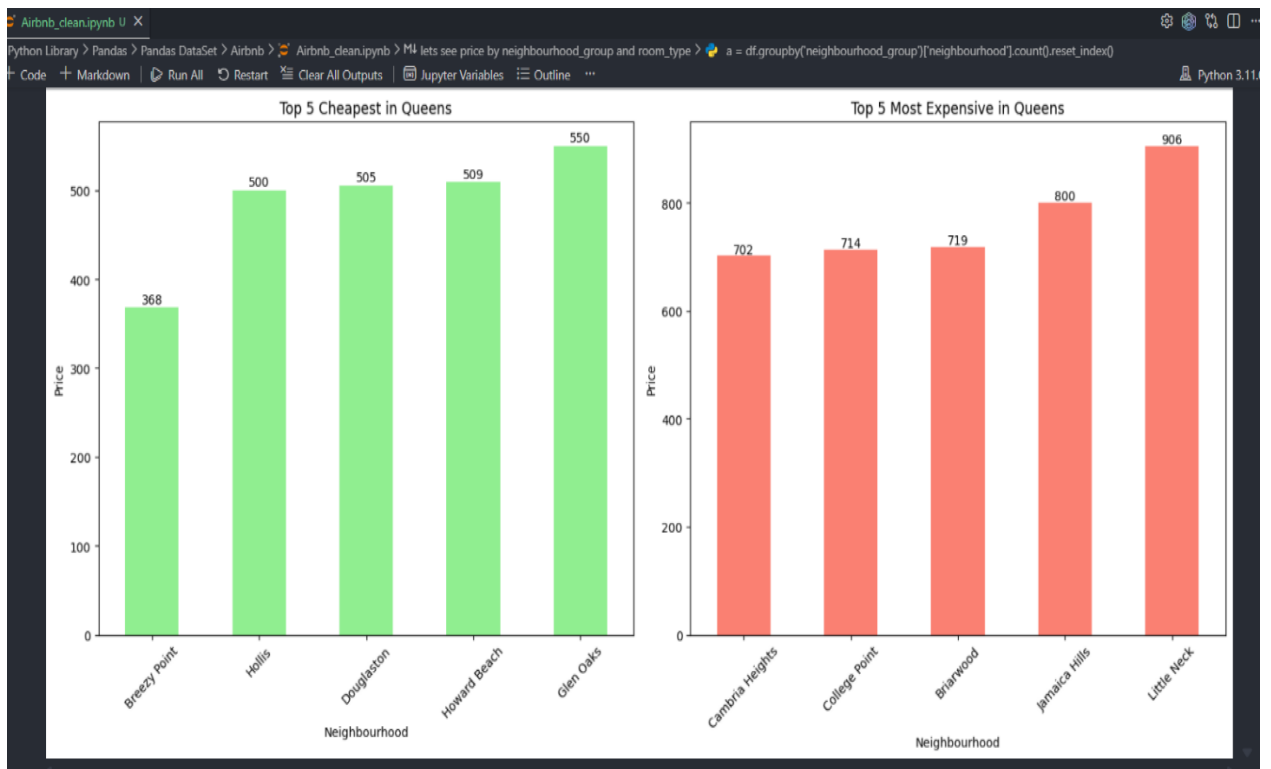
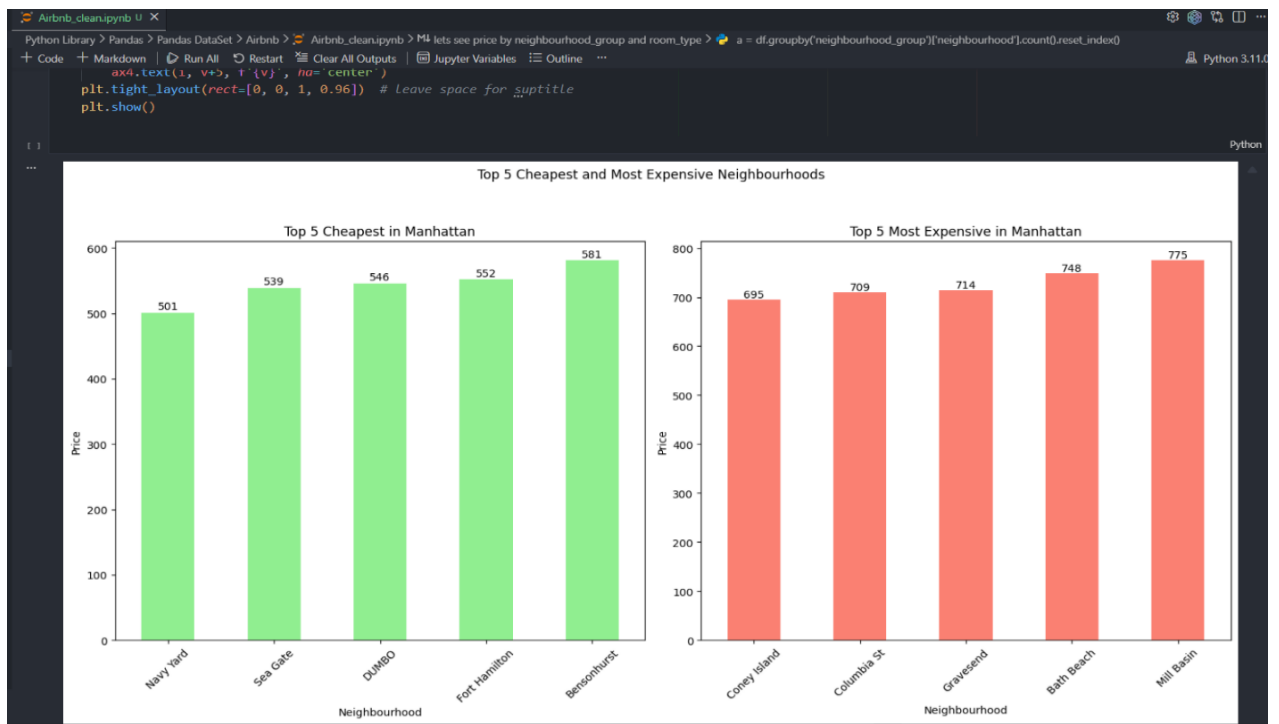
```

Airbnb_cleaningynb
Python Library > Pandas > Pandas Dataset > Airbnb > Airbnb_cleaningynb > P4 lets see price by neighbourhood group and room type > #Che
Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline
Cheapest neighbourhood group: Manhattan
Most expensive neighbourhood group: Queens
-----
Top 5 cheapest neighbourhoods in Manhattan:
neighbourhood
Navy Yard 501
Sea Gate 539
DUMBO 546
Fort Hamilton 552
Bensonhurst 581
Name: price, dtype: int64

Top 5 most expensive neighbourhoods in Manhattan:
neighbourhood
Coney Island 695
Columbia St 709
Gravesend 714
Bath Beach 748
Mill Basin 775
Name: price, dtype: int64

Top 5 cheapest neighbourhoods in Queens:
neighbourhood
Breezy Point 368
Hollis 500
...
Briarwood 719
Jamaica Hills 800
Little Neck 906
Name: price, dtype: int64
Output is truncated. View as a scrollable element or open in a te

```

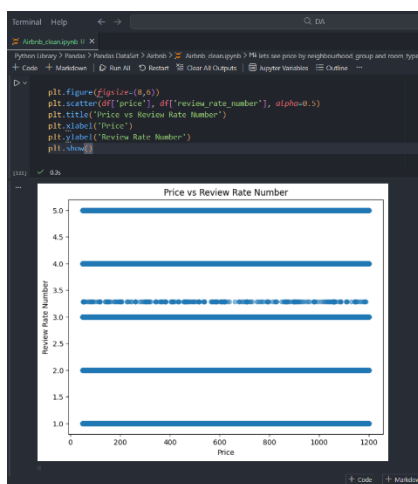


2. Review Behaviour Analysis

A. Review rate vs Price (Does price affect reviews?)

```
df['price'].corr(df['review_rate_number'])  
  
np.float64(-0.006491159161648853)
```

Returns a value between -1 and 1
 >0 → positive correlation
 <0 → negative correlation
 ~ 0 → no linear correlation



Insight :

- Correlation ≈ 0
- Higher price does not guarantee better

Now lets label price into 4 bins :

Low : \$50 - \$340
Medium : \$341 - \$625
High : \$625 - \$913
Very High : \$914 - \$1200



```
# Define Labels for the bins  
labels = ['Low', 'Medium', 'High', 'Very High']  
  
# Map the bins to labels  
df['price_label'] = pd.cut(df['price'], 4, labels=labels)  
price_review = df.groupby('price_label')['review_rate_number'].mean()  
price_review  
  
# '''  
# Insight:  
# Higher price does NOT guarantee higher reviews.  
# '''  
  
price_review = df.groupby('price_label')['review_rate_number'].mean()  
  
price_review  
Low      3.278113  
Medium   3.302382  
High      3.279617  
Very High 3.255713  
Name: review_rate_number, dtype: float64
```

Insight :

High price doesn't affect review

B. Reviews per month vs Price (Are cheaper listings booked more frequently?)

```
b) Reviews per Month vs Price

plt.scatter(df['price'], df['reviews_per_month'], alpha=0.5)
plt.xlabel('Price')
plt.ylabel('Reviews per Month')
plt.show()

''' This answers:
Are cheaper listings booked more frequently?
'''
```

We don't see some serious change in review per month in relation with price.

However ,Cheaper listings get more bookings.



C. Total number of Review by Room type (which room type do have more review?)

```
Number of review by room type

room_type_review = df.groupby('room_type')['number_of_reviews'].sum().sort_values()
room_type_review

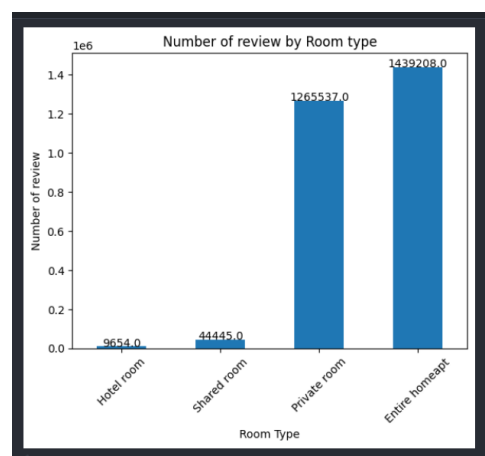
''' this answers :
Which room type do have more review?
'''

''' this answers : \nWhich room type do have more review\n'

room_type_review.plot(kind='bar')
plt.title('Number of review by Room type')
plt.xlabel('Room Type')
plt.ylabel('Number of review')
plt.xticks(rotation=45)

for i,v in enumerate(room_type_review):
    plt.text(i,v , f'{v}', ha='center')

plt.show()
```



Insight : Entire homes receive most

3. Time – Bases analysis

A. Listings by Construction Year

```
year_const = df['year'].value_counts().sort_index()

year_const.plot(kind='line')

plt.title('Listing by construction year')
plt.xlabel('Year')
plt.ylabel('Count')
plt.xticks(rotation=45)

for x, y in zip(year_const.index, year_const.values):
    plt.text(x, y, f'{y}', ha='center', va='bottom')

plt.show()
```



There is no any such significant trend showing increase in newer properties

In 2006 : Most number of properties built

In 2013 : Least number of properties built

B. Proper Age vs Price

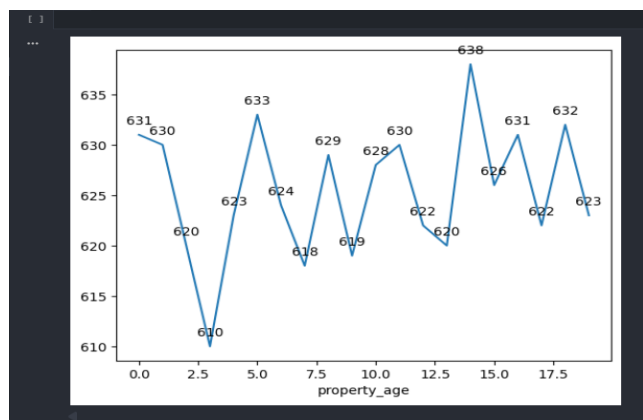
```
b) New vs Old Properties Pricing

df['property_age'] = df['year'].max() - df['year']
price_year = df.groupby('property_age')['price'].mean().astype(int)

...
Insight:
Do Newer properties often charge higher prices.
...

price_year.plot(kind='line')

for i, v in enumerate(price_year):
    plt.text(i, v+1, f'{v}', ha='center')
```



As we can prices are up down , irrespective of the property age ,

Which concludes price is almost same for all property of all ages, As we can see there is uptrend from last 2-3 years property age which also gives hint that new properties are increasing prices.

Most Important Insights from Airbnb Data Analysis

1 Pricing Insights

- **Location dominates price:** Listings in Queens are the most expensive; Manhattan are cheaper.
- **Room type affects price:** Entire homes/apartments are generally higher in count but not always the most expensive due to competition.
- **Property age matters slightly:** Newer properties charge slightly higher prices than older ones.

2 Guest Behavior & Reviews

- **Higher price $\neq$ better reviews:** No correlation between price and review ratings.
- **Cheaper listings are booked more:** Lower-priced properties tend to have higher reviews per month, indicating more frequent bookings.
- **Popular room type:** Entire home/apartment listings receive the highest number of reviews.

3 Location-Based Insights

- **Micro-markets exist:** Even within the same neighbourhood group, some neighborhoods are significantly more expensive than others.
- **Top 5 cheapest vs most expensive neighborhoods:** Identified for both cheapest and most expensive groups (enabling location-specific pricing strategy).

5 Supply & Market Trends

- **Newer constructions are slowly increasing:** Airbnb listings are concentrated in newer properties, indicating market growth.

6 Operational Insights

- **Data cleaning matters:** Proper handling of missing values, duplicates, and text normalization improves reliability of insights.
- **Critical features for analysis:** Price, room type, neighbourhood group, property age, reviews, and host listings count are key determinants of Airbnb performance metrics.

Overall Business Takeaways

1. **Location & room type drive pricing decisions.**
2. **Price alone does not improve guest satisfaction.**
3. **Cheaper listings see higher booking frequency.**
4. **Newer properties are slightly high in prices,**
5. **Neighborhood-level insights enable targeted pricing and investment strategies.**

Airbnb Business Suggestions

1. **Dynamic Pricing Strategy**
 - Adjust prices based on neighborhood, room type, and property age to maximize revenue.
2. **Focus on Guest Experience**
 - Improve amenities and service quality, since higher price does not guarantee better reviews.
3. **Promote Affordable Listings**
 - Highlight lower-priced listings to increase booking frequency and occupancy.
4. **Encourage Short-Term Stays**
 - Listings with shorter minimum nights attract more bookings; promote them in search results.
5. **Neighborhood-Specific Strategies**
 - Identify micro-markets and set location-based pricing and marketing strategies.
6. **Support Hosts with Multiple Listings**
 - Provide analytics tools to professional hosts for better pricing and property management.
7. **Invest in Newer Properties**
 - Encourage investment or upgrades in newer properties to allow premium pricing.
8. **Monitor Extreme Price Outliers**
 - Avoid skewing market perception and provide guidance for luxury listings.
9. **Market Expansion Focus**
 - Expand in neighborhoods with increasing supply and high growth potential.
10. **Data-Driven Host Guidance**
 - Provide hosts with insights on demand trends, booking patterns, and competitive pricing.

Conclusion

This project demonstrates a complete end-to-end analysis of Airbnb listings, starting from data cleaning and preprocessing to feature engineering and exploratory data analysis. Through systematic handling of missing values, duplicates, and unstructured data, the dataset was transformed into a reliable, analysis-ready format.

The analysis revealed key business insights: location and room type are the strongest drivers of price, newer properties command a slight premium, cheaper listings are booked more frequently, and higher prices do not necessarily lead to better reviews. Neighborhood-level analysis identified micro-markets, helping to guide pricing strategies and investment decisions. Host behavior analysis showed that a few hosts with multiple listings significantly influence the market.

Overall, this project provides actionable insights for pricing strategies, guest satisfaction improvement, market segmentation, and investment decisions, demonstrating the value of data-driven decision-making in the short-term rental market.